Experiment 4: Implementation of Class Diagram

Order Management System

---

1. Question

Write a Java program to implement a class diagram for an Order Management System using inheritance. Create classes order, specialorder, normalorder, and customer. Demonstrate order sending, dispatching, receiving, and closing operations.

2. Steps

1. Start the program.
2. Create an order class with private data members date and number.
3. Create a constructor to initialize the order details.
4. Define confirm() and close() methods in the order class.
5. Create a specialorder class that inherits from order.
6. Define the dispatch() method for a special order.
7. Create a normalorder class that inherits from order.
8. Define dispatch() and receive() methods for a normal order.
9. Create a customer class with private members name and location.
10. Create the sendorder() method to send an order.
11. Create the receiveorder() method to receive and close an order.
12. Create a customer object in the main() method.
13. Create a specialorder object and perform send, dispatch, and receive operations.
14. Create a normalorder object and perform receive, dispatch, and customer receive operations.
15. Display all operations.
16. Stop the program.

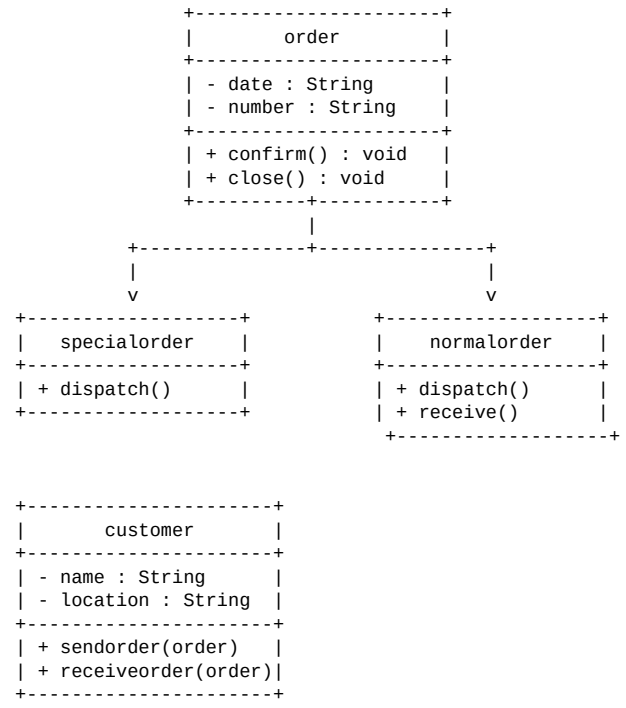
3. Algorithm

1. Start.
2. Define the order class with date and number.
3. Define the constructor, confirm(), and close() methods.
4. Define specialorder as a child class of order.
5. Define the dispatch() method for specialorder.
6. Define normalorder as another child class of order.
7. Define dispatch() and receive() methods for normalorder.
8. Define the customer class with name and location.
9. Define sendorder() and receiveorder() methods.
10. Create customer Seshu from vizag.
11. Create a special order and send it.
12. Dispatch the special order.
13. Receive the special order and close it.
14. Create a normal order.
15. Receive the normal order.
16. Dispatch and receive the normal order.
17. Receive the normal order again and close it.

18 Stop.

4. UML Diagram

UML Class Diagram



UML Relationship

```
specialorder ----------|> order
normalorder  ----------|> order

customer -------------------> order
       uses order object
```

5. Source Code

```
package packagecodejava;

class order {
    private String date;
    private String number;

    public order(String date, String number) {
        this.date = date;
        this.number = number;
    }

    public void confirm() {
        System.out.println("order " + number + " confirm on " + date);
    }

    public void close() {
        System.out.println("order " + number + " closed");
    }
}

class specialorder extends order {
    public specialorder(String date, String number) {
        super("date", "number");
    }
    public void dispatch() {
```

```
        System.out.println("specialorder is " + "dispatched");
    }
}

class normalorder extends order {
    public normalorder(String date, String number) {
        super("date", " number");
    }

    public void dispatch() {
        System.out.println("normalorder is " + "dispatched");
    }

    public void receive() {
        System.out.println("normalorder is" + "received by customer");
    }
}

class customer {
    private String name;
    private String location;

    public customer(String name, String location) {
        this.name = name;
        this.location = location;
    }

    public void sendorder(order order) {
        System.out.println(name + "from" + location + "sent an order");
    }

    void receiveorder(order order) {
        System.out.println(name + "received order");
        order.close();
    }
}

public class Week04 {
    public static void main(String[] args) {

        customer c = new customer("Seshu", "vizag");
        specialorder so = new specialorder("12th august", "44");
        c.sendorder(so);
        so.dispatch();
        c.receiveorder(so);

        System.out.println();

        normalorder no = new normalorder("13th august", "82");
        c.receiveorder(no);
        no.dispatch();
        no.receive();
        c.receiveorder(no);
    }
}
```

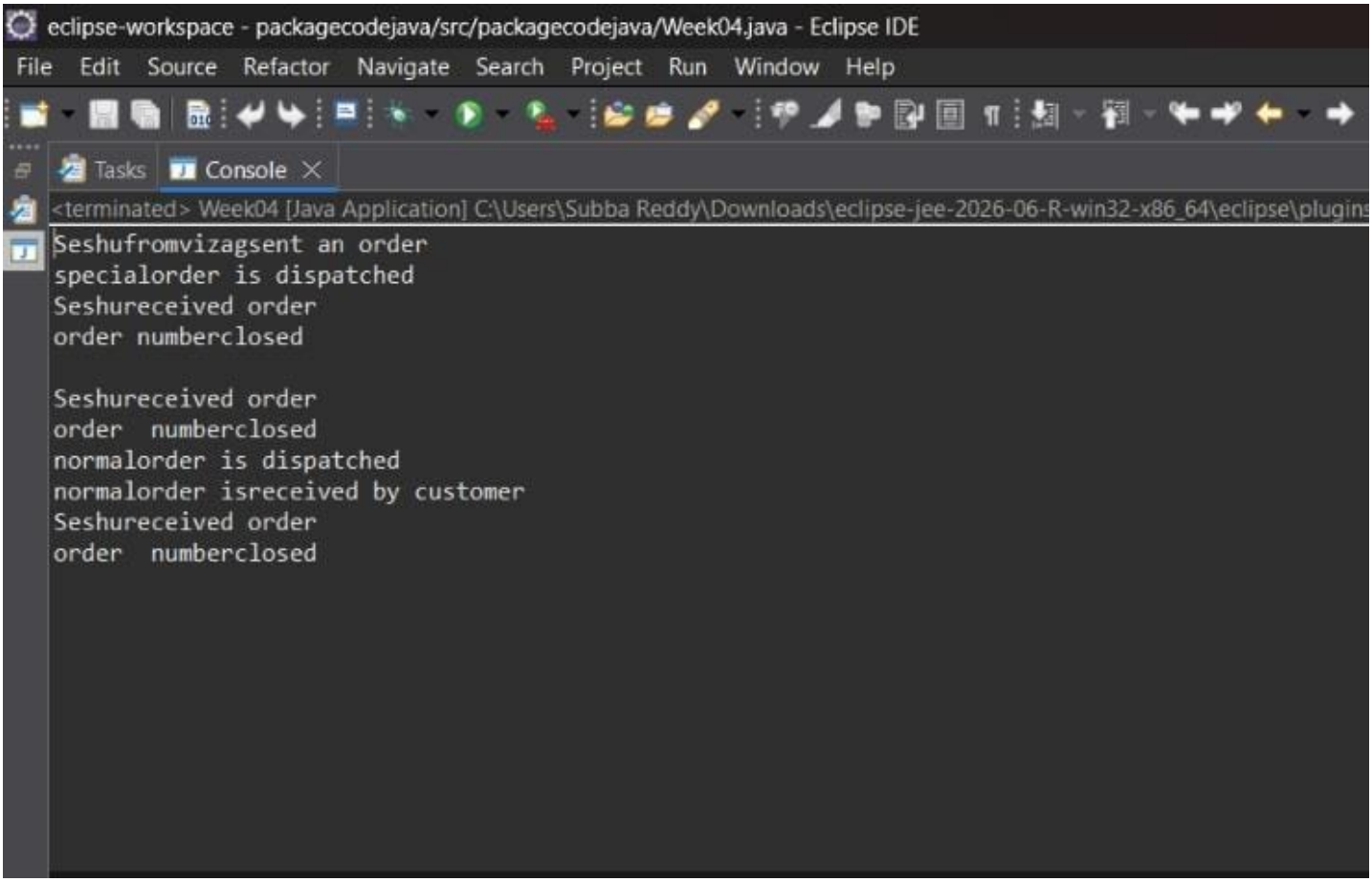
Compilation and Execution

```
javac Week04.java
java packagecodejava.Week04
```

6. Output

```
Seshufromvizagsent an order
specialorder is dispatched
Seshu received order
order numberclosed

Seshu received order
```


eclipse-workspace - packagecodejava/src/packagecodejava/Week04.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Tasks
Console
<terminated> Week04 [Java Application] C:\Users\Subba Reddy\Downloads\eclipse-jee-2026-06-R-win32-x86_64\eclipse\plugins
Seshufromvizagsent an order
specialorder is dispatched
Seshureceived order
order numberclosed
Seshureceived order
order numberclosed
normalorder is dispatched
normalorder isreceived by customer
Seshureceived order
order numberclosed